

CSE 256 | Lecture 4: Linear Models - Optimization

Ndapa Nakashole

January 15, 2026

Recap: Softmax, Loss Function, Regularization

Softmax

$$s = f(x_i; W) = Wx_i$$

Want to interpret raw classifier scores as probabilities

$$P(Y = k \mid X = x_i) = \frac{e^{s_k}}{\underbrace{\sum_j e^{s_j}}_{\text{Softmax}}}$$

Loss Function

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

Softmax loss (Cross-Entropy):

$$L_i = -\log p(y_i \mid x_i; W)$$

Today

- ① Optimization intuition: loss landscapes
- ② Numerical gradients: approximating derivatives
- ③ Analytic gradients: softmax loss (*text classification*)
- ④ Gradient descent: putting it all together
- ⑤ Backpropagation (*just the definition*)

Optimization: Landscape Intuition

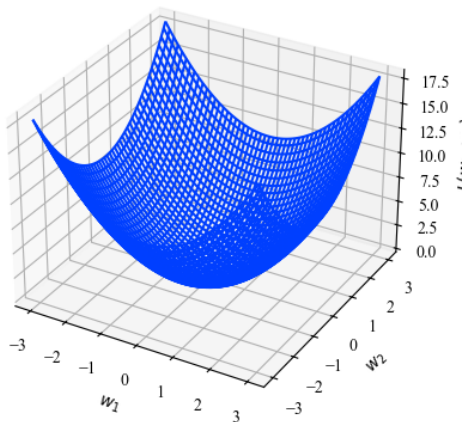
- ▶ Think of the loss as defining a **landscape** over parameters $\mathbf{W}$
 - ⊙ Each point in the landscape corresponds to one possible setting of $\mathbf{W}$
- ▶ The **height** of the landscape is the loss: High points (hills) → the model is doing poorly; Low points (valleys) → the model is doing well
- ▶ **Learning**: move through the landscape to reduce the loss



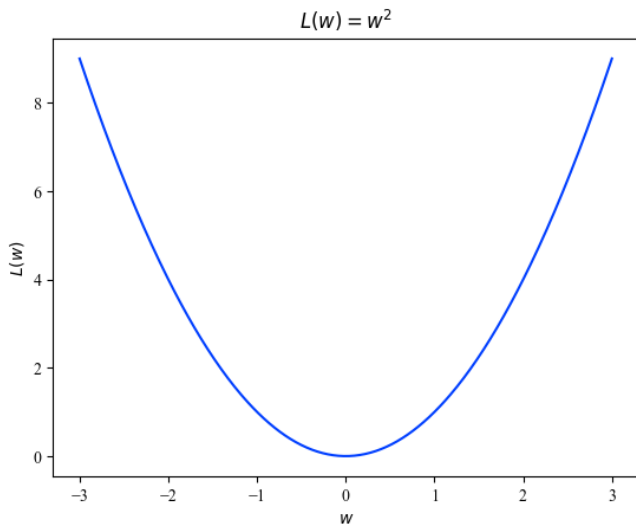
Optimization: Simple 2D Function

- ▶ Input to the loss: two parameters (w_1, w_2) ; Output: a single number (the loss value)
- ▶ Every point (w_1, w_2) has a loss value

$$L(w_1, w_2) = w_1^2 + w_2^2$$



Optimization: Even Simpler function



Strategy #1: Random Search

- ▶ Treat loss function $L(\mathbf{W})$ as a black box (we are blindfolded)
- ▶ Can evaluate the loss at any chosen point (altitude sensor)
- ▶ Randomly sample different parameter settings (jump around)
- ▶ Keep track of the lowest loss seen so far



Strategy #2: Iterative Improvement

- ▶ Finding the globally best weights $\mathbf{W}$ directly is hard
- ▶ Key idea: instead of searching everywhere, **start somewhere and improve step by step**
 - ◉ Initialize $\mathbf{W}$ randomly
 - ◉ Iteratively update $\mathbf{W}$ to **reduce the loss** $L(\mathbf{W})$
- ▶ Key question:
Given the current $\mathbf{W}$, which direction should we move to reduce the loss?

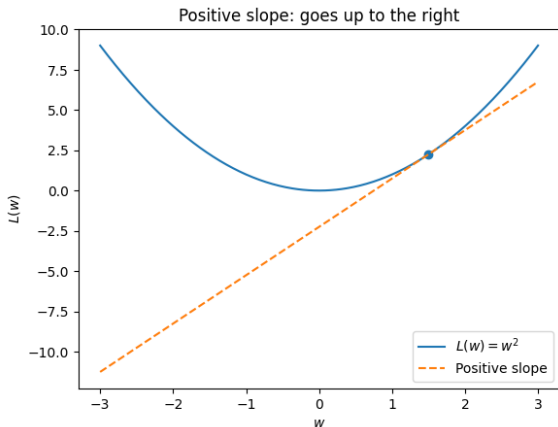
Strategy #2: Iterative Improvement: taking the next step



- ① **Look Around (Locally):**
Which way goes uphill *right here?* (math: compute the gradient at the current point)
- ② **Take a Small Step:** Move *opposite* the gradient (downhill) (step size = learning rate)
- ③ **Repeat and Re-evaluate:**
After each step, recompute the gradient and continue descending

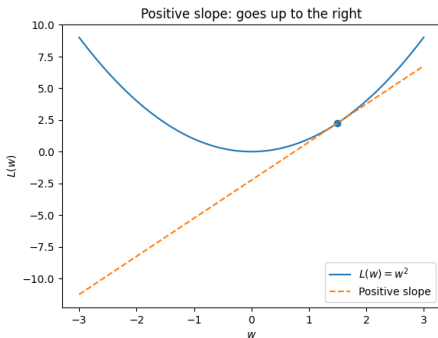
Slope (1D intuition): simple function $L(w) = w^2$

- ▶ The **slope** indicates the direction and rate of change of a function.
- ▶ To go **downhill**, we move in the *opposite direction of the slope*
 - ⊙ This is gradient descent in one dimension

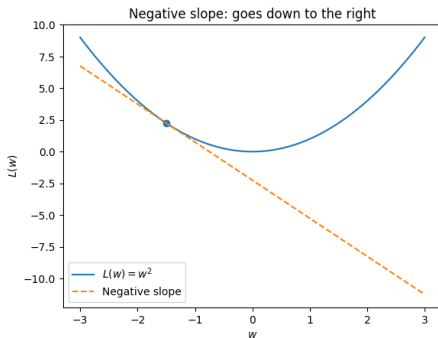


Slope Intuition: Positive vs. Negative

Positive slope, function increases



Negative slope, function decreases



Numerical Gradients

Finite Difference and the Derivative

- ▶ The derivative measures the **slope** of a function at a point
- ▶ **In 1 dimension:** the derivative is defined as

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In practice: we approximate this using a small but finite h (*finite differences*)

In multiple dimensions:

- ▶ The **gradient** is a vector of partial derivatives
- ▶ It gives the direction of *steepest increase* of the function. The gradient's magnitude is the steepness

Numerical gradient in multiple dimensions: 1st dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,
?,...]

Numerical gradient in multiple dimensions: 1st dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,
?,...]

Numerical gradient in multiple dimensions: 1st dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[-2.5,
?,
?,

$$(1.25322 - 1.25347)/0.0001 = -2.5$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,
?,...]

► In the first dimension the gradient is **negative**.

- ◉ Increasing this weight decreases the loss.
- ◉ Gradient descent therefore moves **opposite the gradient**, i.e., increases this weight.

2nd dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25353

gradient dW:

[-2.5,
?,
?,
?,
?,
?,
?,
?,
?,...]

2nd dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25353

gradient dW:

[-2.5,
0.6,
?,
?,

$$(1.25353 - 1.25347)/0.0001 = 0.6$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,...]

► In the second dimension the gradient is **positive**.

- ◉ Increasing this weight increases the loss.
- ◉ Gradient descent therefore moves **opposite the gradient**, i.e., decreases this weight.

3rd dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
?,
?,
?,
?,
?,
?,
?,...]

But Numerical Gradients Don't Scale ...

current W:	W + h (third dim):	gradient dW:
[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,...]	[0.34, -1.11, 0.78 + 0.0001 , 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,...]	[-2.5, 0.6, 0 , ?, ?,...]
loss 1.25347	loss 1.25347	Numeric Gradient - Slow! Need to loop over all dimensions - Approximate

- ▶ Numerical gradients require looping over *every* parameter
 - ⊙ Infeasible for models with many weights
 - ⊙ Approximate
- ▶ Instead: use calculus to derive an **analytic gradient**
- ▶ ⊙ Can then compute the **entire gradient vector at once**

Analytic Gradients

Analytic Gradient for Softmax Loss

Goal: compute the gradient of the loss with respect to the weights

$$\nabla_W L(W)$$

Recall: Softmax loss

$$L(W) = - \sum_{i=1}^n \log p(y_i \mid x_i; W)$$

- ▶ Numerical gradients are slow and approximate
- ▶ Analytic gradients are fast and exact (up to floating-point)

Analytic Gradient: for Softmax Loss

This is the function we will differentiate.

$$L(W) = - \sum_{i=1}^n \log p(y_i \mid x_i; W)$$

Full expression for the loss: (*Walk through derivation at home.*)

$$\begin{aligned} L(\mathbf{W}) &= - \sum_{i=1}^n \log p(y_i \mid x_i; \mathbf{W}) \\ &= - \sum_{i=1}^n \log \left(\frac{\exp(\mathbf{W}_{y_i} \cdot x_i)}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{W}_{y'} \cdot x_i)} \right) \\ &= - \sum_{i=1}^n \left(\mathbf{W}_{y_i} \cdot x_i - \log \sum_{y' \in \mathcal{Y}} \exp(\mathbf{W}_{y'} \cdot x_i) \right) \end{aligned}$$

Analytic Gradient: Recall the Softmax Loss

This is the function we will differentiate.

$$\begin{aligned} L(W) &= - \sum_{i=1}^n \log p(y_i \mid x_i; W) \\ &= - \sum_{i=1}^n \left(W_{y_i} \cdot x_i - \log \sum_{y' \in \mathcal{Y}} \exp(W_{y'} \cdot x_i) \right) \end{aligned}$$

- ▶ Built from simple pieces: dot products, exp, sum, log
- ▶ We will apply the chain rule to these pieces

Gradient of the Loss Function (1/3)

Loss for a single training example:

$$L_i(W) = - \left(W_{y_i} \cdot x_i - \log \sum_{y' \in \mathcal{Y}} \exp(W_{y'} \cdot x_i) \right)$$

Step 1: Differentiate the first term

$$\frac{\partial}{\partial W_{y_i}} (W_{y_i} \cdot x_i) = x_i$$

Gradient of the Loss Function (2/3)

Step 2: Differentiate the second term

$$\log \sum_{y' \in \mathcal{Y}} \exp(W_{y'} \cdot x_i)$$

Apply the chain rule

$$\frac{d}{dz} \log z = \frac{1}{z}$$

$$\frac{\partial}{\partial W_{y_i}} \log \sum_{y' \in \mathcal{Y}} \exp(W_{y'} \cdot x_i) = \frac{1}{\sum_{y'} \exp(W_{y'} \cdot x_i)} \cdot \frac{\partial}{\partial W_{y_i}} \sum_{y'} \exp(W_{y'} \cdot x_i)$$

Key observation:

Only the term with $y' = y_i$ depends on W_{y_i}

$$\frac{\partial}{\partial W_{y_i}} \sum_{y'} \exp(W_{y'} \cdot x_i) = \exp(W_{y_i} \cdot x_i) x_i$$

Gradient of the Loss Function (3/3)

Combining the results

$$\frac{\partial L_i(W)}{\partial W_{y_i}} = -x_i + \frac{\exp(W_{y_i} \cdot x_i)}{\sum_{y' \in \mathcal{Y}} \exp(W_{y'} \cdot x_i)} x_i$$

Recognize the softmax probability

$$\boxed{\frac{\partial L_i(W)}{\partial W_{y_i}} = -x_i + p(y_i \mid x_i; W) x_i}$$

- ▶ For the correct class: pull weights toward the input (make the weights look more like the example), and decrease them in proportion to how confident the model already is.
- ▶ For other classes: update is similar but without the first term

Why Linear Classifiers Learn Templates

Gradient descent makes each class weight vector look more like the training examples of that class.

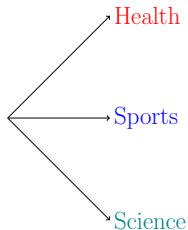
- ▶ Each row of W acts as a **template** for a class
- ▶ When the model is **wrong**:
 - ◉ $p(y_i | x_i; W) \approx 0$
 - ◉ Gradient $\approx -x_i$, update pulls W_{y_i} **toward** x_i
- ▶ When the model is **confident and correct**:
 - ◉ $p(y_i | x_i; W) \approx 1$
 - ◉ Gradient ≈ 0 , very small or no update

Interpretation: Training builds class templates by nudging weights toward examples they should recognize.

Example with Text Classification

Text Document:

too many drug trials, too few patients



Feature representation: Does the text mention drugs, patients, or baseball?

$$x_i = [\mathbb{I}(\text{drug}), \mathbb{I}(\text{patients}), \mathbb{I}(\text{baseball})] = [1, 1, 0]$$

Example with Text Classification: Gradients

Gradient for the correct class

$$\frac{\partial L_i(W)}{\partial \mathbf{W}_{y_i}} = -x_i + p(y_i \mid x_i; W) x_i$$

Gradient for other classes

$$\frac{\partial L_i(W)}{\partial \mathbf{W}_{y'}} = p(y' \mid x_i; W) x_i$$

Example

"too many drug trials, too few patients"

True label: $y_i = \text{Health}$

$x_i = [1, 1, 0]$

$P(y \mid x_i) = [\text{0.2}, \text{0.5}, \text{0.3}]$ (illustrative)

Interpretation:

- ▶ Pull **Health** weights toward the example
- ▶ Push other class templates away
- ▶ Strength proportional to how wrong the model

$$\nabla \mathbf{W}_{\text{health}} = -[1, 1, 0] + 0.2[1, 1, 0]$$

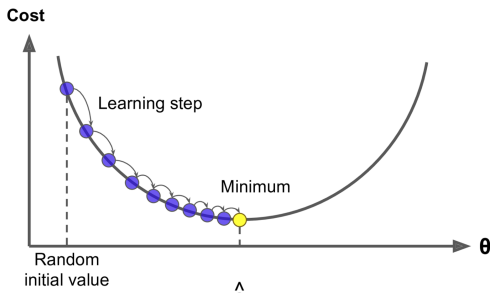
$$\nabla \mathbf{W}_{\text{sports}} = 0.5[1, 1, 0]$$

$$\nabla \mathbf{W}_{\text{science}} = 0.3[1, 1, 0]$$

Putting it all together: Gradient Descent: Iterative Improvement

- ▶ **Goal:** minimize the loss function $L(\mathbf{W})$
- ▶ **Key idea:** start from an initial guess of the parameters $\mathbf{W}$
 - ⊙ Compute the gradient $\nabla_{\mathbf{W}} L(\mathbf{W})$
 - ⊙ Take a small step in the direction of **steepest descent**
- ▶ Repeat until the loss stops decreasing (convergence)

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} L(\mathbf{W})$$



Minibatch Gradient Descent

Recall: loss of entire dataset

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Gradient of the loss:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(f(x_i, W), y_i)$$

Problem: computing this sum is expensive when N is large

Minibatch Gradient Descent

- ▶ Use a small **minibatch** instead of all N examples
- ▶ Approximate the full gradient
- ▶ Common batch sizes: 32, 64, 128, 256
- ▶ Stochastic Gradient Descent (SGD): batch size = 1

Minibatch Gradient Descent

```
# Vanilla Minibatch Gradient Descent
while True:
    data_batch = sample_training_data(data, 256) # sample 256 examples
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
    weights += - step_size * weights_grad # perform parameter update
```

Backpropagation

- ▶ **Gradient Descent** An optimization procedure: repeatedly compute gradients and update parameters to reduce the loss.
- ▶ **Backpropagation** An efficient algorithm to compute gradients of the loss with respect to *all* model parameters.
 - ◉ Based on the **chain rule of calculus**
 - ◉ Reuses intermediate computations

In this course, we focus on understanding what gradients mean. The mechanics of backpropagation are handled for us by automatic differentiation (autograd).

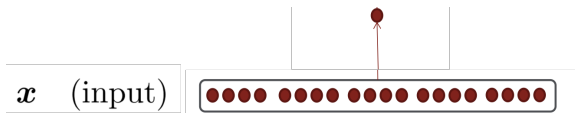
Text Classification with Linear Models: Big Picture

► Represent text as numbers

- Convert raw text into a feature vector x
- Each dimension captures the presence or strength of a feature

► Score each class with a linear model

- Compute class scores: $s = Wx$
- Convert scores to probabilities (softmax)
- Pick the class with highest probability



► Training choices (hyperparameters): regularization strength, learning rate, batch size, number of iterations

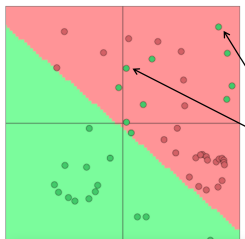
Linear Classifiers: Strengths and Limitations

► What linear classifiers do well

- Simple, fast, and interpretable
- Work well when classes are linearly separable

► Key limitation

- Cannot model complex, nonlinear patterns in the data



Next: Feedforward Neural Networks: adding nonlinearity and expressive power

Questions ?